

Annexe — HTTP : le protocole du Web

Notion transversale, placée en annexe car non essentielle pour la journée HTML (on y revient en JS asynchrone et Node). À lire en auto-formation, ou en cours quand le formateur juge utile d'aborder client-serveur.

HTTP EN UNE PHRASE

Le protocole qui définit comment un navigateur (client) et un serveur s'échangent des messages — la base invisible de tout ce qui se passe quand on tape une URL.

 **DURÉE ESTIMÉE** : lecture 30 min, exercice DevTools 30 min

Objectifs du chapitre

À l'issue de ce chapitre, vous serez capable de :

- **Expliquer** le modèle client-serveur et le caractère *stateless* de HTTP.
- **Identifier** les composants d'une URL (protocole, domaine, chemin, paramètres, fragment).
- **Distinguer** les méthodes **GET**, **POST**, **PUT**, **PATCH**, **DELETE** et leurs cas d'usage.
- **Interpréter** les familles de codes de statut HTTP (2xx, 3xx, 4xx, 5xx).
- **Analyser** les requêtes d'une page via les DevTools du navigateur (onglet Network).

EN PRATIQUE — SITUATION PROFESSIONNELLE

Votre application front ne reçoit pas les données attendues. Sans HTTP, impossible de diagnostiquer : est-ce une erreur 404 (mauvaise URL) ? 401 (pas authentifié) ? 500 (le serveur a planté) ? 200 mais mauvaise réponse (bug front) ? Ce chapitre vous donne le vocabulaire pour lire les DevTools et pointer la cause.

Sommaire

- **A. Modèle client-serveur**
- **B. Anatomie d'une URL**
- **C. Méthodes HTTP**
- **D. Codes de statut**
- **E. En-têtes HTTP**
- **F. Pipeline de rendu du navigateur**

A. Modèle client-serveur

1. Principe

HTTP (*HyperText Transfer Protocol*) est un protocole **client-serveur** : un navigateur (**client**) envoie une **requête**, un serveur renvoie une **réponse**.



2. Caractéristiques

- **Stateless** (sans état) — chaque requête est indépendante, le serveur ne *se souvient* pas de la précédente. Les sessions sont gérées via des cookies ou tokens.
- **Indépendant du type de données** — HTTP peut transporter du HTML, du JSON, des images, de la vidéo, etc.
- **Textuel** (HTTP/1.1) ou **binaire** (HTTP/2 et HTTP/3) selon la version.

3. Ports standards

Protocole	Port	Usage
HTTP	80	Version non sécurisée (quasi-abandonnée)
HTTPS	443	Version chiffrée via TLS — standard actuel

⚠ HTTPS obligatoire aujourd'hui

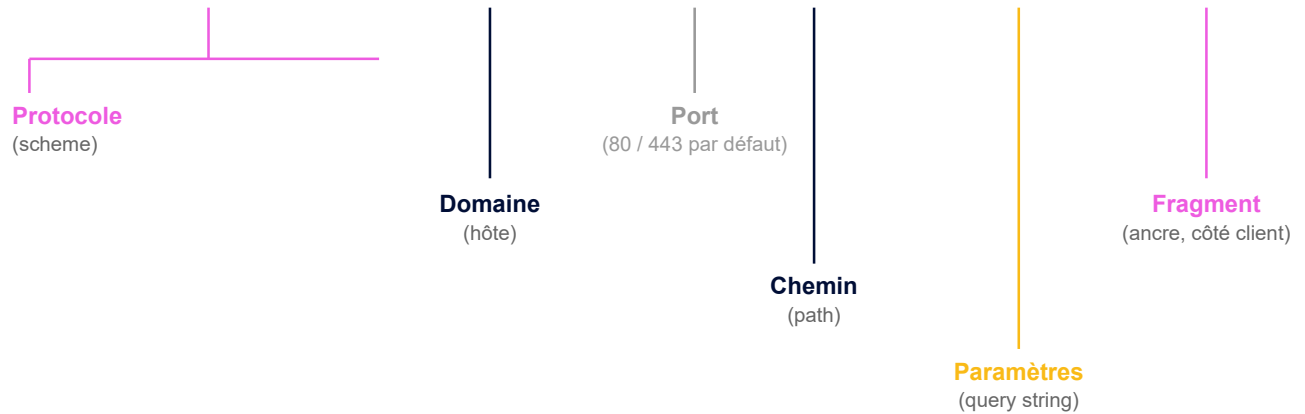
HTTPS est devenu **indispensable** pour :

- la sécurité des données (chiffrement en transit) ;
- la confiance utilisateur (les navigateurs marquent HTTP comme *non sécurisé*) ;
- le référencement (Google pénalise les sites en HTTP).

Tout site en production en 2026 **doit** être servi en HTTPS.

B. Anatomie d'une URL

https://www.example.com:443/blog/article?id=42&lang=fr#section-2



Chaque composant

Élément	Exemple	Rôle
Protocole	https://	Protocole utilisé pour la communication
Domaine	www.example.com	Nom d'hôte qui identifie le serveur
Port	:443	Port TCP (implicite : 80 HTTP / 443 HTTPS)
Chemin	/blog/article	Ressource demandée sur le serveur
Paramètres	?id=42&lang=fr	Arguments (query string), séparés par &
Fragment	#section-2	Ancre dans la page, traité côté client seulement

C. Méthodes HTTP

1. Les 5 méthodes principales

Méthode	Intention	Cas d'usage	Idempotente ?
GET	Lire une ressource	Afficher une page, récupérer des données	✓ Oui
POST	Créer une ressource	Soumettre un formulaire, créer un utilisateur	✗ Non
PUT	Remplacer une ressource	Mise à jour complète d'une entité	✓ Oui
PATCH	Modifier partiellement	Mise à jour d'un seul champ	✗ (dépend)
DELETE	Supprimer une ressource	Supprimer un utilisateur	✓ Oui

Idempotence

Une méthode est **idempotente** si appliquer la même requête N fois donne le même résultat qu'une seule fois. `GET /users/42` appelé 3 fois renvoie toujours le même utilisateur. `POST /users` appelé 3 fois crée 3 utilisateurs.

2. Autres méthodes utiles

- `HEAD` — comme `GET`, mais renvoie seulement les en-têtes (vérifier qu'une ressource existe sans la télécharger).
- `OPTIONS` — utilisé pour les requêtes CORS (*preflight*).

D. Codes de statut

Chaque réponse HTTP commence par un code à 3 chiffres qui indique son issue.

1. Les 5 familles

Famille	Sens	Exemples
1xx	Information	100 Continue
2xx	Succès	200 OK , 201 Created , 204 No Content
3xx	Redirection	301 Moved Permanently , 302 Found , 304 Not Modified
4xx	Erreur client	400 Bad Request , 401 Unauthorized , 403 Forbidden , 404 Not Found , 429 Too Many Requests
5xx	Erreur serveur	500 Internal Server Error , 502 Bad Gateway , 503 Service Unavailable

2. Codes à connaître par cœur

Code	Signification	Quand
200	OK	Tout va bien
201	Created	Ressource créée (réponse typique d'un POST)
301	Moved Permanently	L'URL a changé définitivement
304	Not Modified	Pas besoin de re-télécharger, utilise le cache
400	Bad Request	La requête est malformée
401	Unauthorized	Pas authentifié
403	Forbidden	Authentifié mais pas les droits
404	Not Found	Ressource inexistante
500	Internal Server Error	Plantage serveur

♦ 401 vs 403

- **401** : *Je ne sais pas qui tu es* → il faut se connecter.
- **403** : *Je sais qui tu es, mais tu n'as pas le droit* → l'authentification ne changera rien.

E. En-têtes HTTP

Les **en-têtes** (*headers*) portent des métadonnées pour la requête ou la réponse.

1. En-têtes de requête courants

```
GET /blog/article HTTP/1.1
Host: www.example.com
User-Agent: Mozilla/5.0 ...
Accept: text/html,application/json
Accept-Language: fr-FR
Authorization: Bearer eyJhbGciOi...
Cookie: session=abc123
```

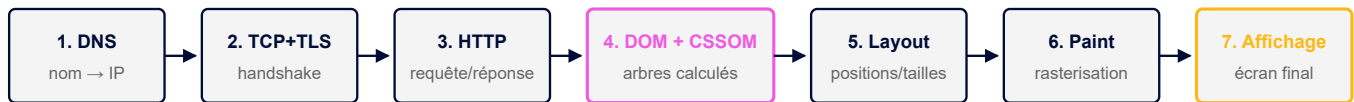
2. En-têtes de réponse courants

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=UTF-8
Content-Length: 4521
Cache-Control: max-age=3600
Set-Cookie: session=abc123; HttpOnly; Secure
```

En-tête	Rôle
Content-Type	Type MIME du contenu (<code>text/html</code> , <code>application/json</code> , <code>image/png</code> , ...)
Content-Length	Taille du contenu en octets
Cache-Control	Durée et conditions de mise en cache
Authorization	Identifiants d'authentification (Bearer token, Basic auth)
Set-Cookie	Pose un cookie sur le navigateur

F. Pipeline de rendu du navigateur

Quand vous tapez une URL et appuyez sur Entrée, voici les étapes (simplifiées) :



Toute latence à une étape retarde toutes les suivantes — d'où l'importance d'optimiser chaque maillon.

1. Résolution DNS

www.example.com → 93.184.216.34

Le navigateur demande à un serveur DNS l'adresse IP du domaine, puis met en cache la réponse.

2. Établissement de la connexion TCP + TLS

- **TCP handshake** en 3 étapes (SYN, SYN-ACK, ACK).
- **TLS handshake** si HTTPS (échange de certificats, négociation des clés de chiffrement).

3. Requête HTTP + réception

Le navigateur envoie la requête et reçoit la réponse (HTML, puis CSS, JS, images, fonts...).

4. Construction du DOM et du CSSOM

- **DOM** (*Document Object Model*) : arbre des balises HTML.
- **CSSOM** (*CSS Object Model*) : arbre des règles CSS calculées.

5. Layout

Calcul des positions et dimensions de chaque élément (où est la boîte ? quelle taille ?).

6. Paint + composition

Rasterisation des pixels, composition des couches (*layers*), affichage à l'écran.

♦ Optimisation du rendu

- `loading="lazy"` sur les images non visibles → moins de requêtes initiales.
- `<link rel="preload">` sur les ressources critiques → chargées en priorité.
- `defer` et `async` sur les `<script>` → évite de bloquer le parsing HTML.
- **Cache** (`Cache-Control`) → évite de re-télécharger les ressources déjà connues.

7. Les DevTools pour observer tout ça

Dans Chrome / Firefox : `F12` → onglet **Network**.

Vous verrez :

- Chaque requête avec sa méthode, son statut, son type MIME, sa taille, son temps de réponse.
- La cascade temporelle (waterfall) — visualise quoi bloque quoi.
- Les en-têtes complets de chaque requête et réponse.

À retenir

- HTTP est un protocole **client-serveur**, **stateless**, qui véhicule requêtes et réponses.
- URL = `protocole://domaine:port/chemin?params#fragment`.
- Méthodes principales : **GET** (lire), **POST** (créer), **PUT** (remplacer), **PATCH** (modifier), **DELETE** (supprimer).
- Familles de codes : **2xx** succès, **3xx** redirection, **4xx** erreur client, **5xx** erreur serveur.
- Codes clés : **200, 201, 301, 400, 401, 403, 404, 500**.
- **HTTPS est obligatoire** en production.
- L'onglet **Network** des DevTools est l'outil n°1 pour diagnostiquer les problèmes.

Pour aller plus loin

- **MDN — Vue d'ensemble de HTTP**
- **MDN — Codes de statut**
- **HTTP Status Codes (référence)**
- **web.dev — Rendering performance**
- **RFC 7231 — HTTP/1.1 Semantics**